

Specification-Driven Architecture

Five Traditions, One Shared Failure Mode

The xtgeoip Project

5 September 2026

Abstract

Deriving a program’s source, documentation and tests from one authoritative specification is an old idea with at least five largely independent lineages: formal refinement, model-driven engineering, interface definition languages, in-project code generation, and the agent-oriented “spec-driven development” that emerged in 2025. This paper surveys the five, reports what is known about how widely each is practised and in which languages, and argues that they share a failure mode that none of their literatures names directly. Derivation guarantees fidelity *downstream* of the specification and guarantees nothing about fidelity *into* it. Because every derived artefact agrees with the specification by construction, a defect at the root produces no disagreement anywhere in the tree — it is instead multiplied across every generated artefact that asserts it. This is structurally the correlated-failure result that Knight and Leveson established for N-version programming in 1986, arrived at from the opposite direction: N-version programming lost independence it assumed it had, whereas a derived tree discards independence deliberately, in exchange for consistency. We illustrate with a defect from the authors’ own codebase in which six generated artefacts agreed on a wrong rule for four months, and the only dissenting voices were two hand-written documents that nothing checked.

1. Introduction

A project that generates its command-line parser, its manual page, its error constants and its test matrix from a single declarative file is doing something that has a name — several names, in fact, belonging to communities that rarely cite one another. The practical question this paper began from was narrow: is this a recognised engineering paradigm, how common is it, and who else does it?

The short answer is that it is a recognised *mechanism* with no single recognised *name*. Five traditions have converged on generating artefacts from an authoritative description, for reasons ranging from mathematical proof to reducing boilerplate in a compiler backend. They differ in what the specification is written in, what is generated from it, which direction the derivation runs, and what guarantee is claimed. What unites them is not a lineage but a structure, and it is the structure that carries the failure mode described in Section 8.

Section 2 states the method and its limits. Section 3 examines what the term denotes today, which is narrower and much more recent than the practice. Sections 4 through 7 survey the traditions, their prevalence, their languages, and a cross-cutting axis — the direction of derivation — that complicates the “single source of truth” slogan. Section 8 develops the shared failure mode and connects it to an established result. Section 9 gives a worked example. Sections 10 and 11 draw practical consequences and identify what the literature does not cover.

2. Method and limitations

This is a survey assembled from public sources over a single working session, not a systematic literature review. It was not conducted with a registered protocol, a defined search string set, or inter-rater agreement, and it makes no claim to exhaustiveness. Where adoption figures are quoted they come from industry surveys with self-selected respondents, whose absolute values should be treated as indicative rather than measured.

One methodological note is worth recording because it affected the survey itself. Search-engine summarisation of this topic proved unreliable in a specific way: a summary offered a confident provenance for the term — tracing it to “1960s NASA workflows” and an “academic formalisation in 2004” — of which only the 2004 citation survived

with the source document. The cited article contains no NASA claim and no discussion of the modern wave at all. Popular writing on specification-driven development is unusually prone to inherited, unsourced history. Every claim of provenance below was checked against a primary document, and where a claim could not be so checked it has been omitted rather than hedged.

The case study in Section 9 is the authors’ own project. It is offered as an existence proof and an illustration of a mechanism, not as evidence of frequency. A single incident in a single small codebase cannot establish how often the failure mode occurs; it can only show concretely what it looks like and why the surrounding machinery did not catch it.

3. What the name denotes

3.1 *The academic thread*

As a term of art, “specification-driven development” has a thin academic record. The encyclopaedic entry for it is a stub which classifies it as a form of documentation-driven development alongside model-driven development, model transformation and round-trip engineering, and which rests on a single substantive citation: Ostroff, Makalsky and Paige’s *Agile Specification-Driven Development* (XP 2004) [1]. That paper is narrower than the name suggests. It argues that tests and contracts are two kinds of specification, useful and complementary, and it makes the argument in Eiffel, where Design by Contract is a language feature. It is a synthesis of test-driven development and contract-based design — not a theory of generating a system from a description.

This matters for anyone searching the literature. The phrase does not retrieve the tradition most practitioners of the mechanism actually belong to. Someone generating a parser from a schema will find almost nothing under this name and a great deal under “model-driven engineering,” “code generation,” “interface definition language” or “language workbench.”

3.2 *The 2025 wave*

Since mid-2025 the term has been claimed, largely successfully, by a different practice: using a written specification as the durable artefact that a large language model generates code from. Amazon’s Kiro (July 2025) structures work into requirements, design and task stages. GitHub’s Spec Kit, open-sourced in September 2025, adds orchestration and a “constitution” of immutable principles. Tessl goes furthest, proposing that the specification rather than the code be the maintained artefact — “spec-as-source.”

The honest current status of this wave is cautious. Thoughtworks placed spec-driven development in the **Assess** ring of its Technology Radar in November 2025 [2] — “worth exploring with the goal of understanding how it will affect your enterprise” — with material reservations:

“the workflows remain elaborate and opinionated. These tools behave very differently depending on task size and type; some generate lengthy spec files that are hard to review ... when they produce PRDs or user stories, it’s sometimes unclear who their intended user is.”

It would be an error — the easiest error this subject invites — to read that assessment as applying to the other four traditions, or to read their maturity as applying to it. A tool ecosystem eight months old and a code generator with thirty years of production service in a compiler are not the same proposition merely because both begin from a file.

4. Five traditions

The following table summarises the five. The rows are ordered by age, not by importance or prevalence.

Tradition	Specification is	Generated	Guarantee claimed
Formal refinement	A logic or ISA description language	Provers, simulators, tests	Machine-checked correctness
Model-driven engineering	A model or a bespoke DSL	Application code, schemas	Productivity, consistency
Interface definition	A wire or API schema	Clients, servers, documentation	Interoperability
In-project generation	A build-time DSL	Source, documentation, tables	Non-duplication
Agent-oriented SDD	Structured prose	Code, tests, documentation	Intent capture (under assessment)

4.1 Formal specification and refinement

The oldest and most demanding tradition writes the specification in a logic and proves the implementation refines it. seL4 is the canonical systems result: a microkernel whose functional correctness is proved in Isabelle/HOL through two refinement steps, from an abstract specification to an executable model to C [3]. CompCert occupies the same ground for a compiler, in Coq.

Instruction-set architecture is where this tradition has quietly become industrial. ARM publishes its architecture in machine-readable form, written in its own ASL specification language — executable, and tested with the same conformance suites ARM uses on silicon [4]. RISC-V has standardised an executable formal specification of its ISA semantics in Sail [5], and Sail models exist for ARMv8-A (derived automatically from the ARM-internal specification), MIPS and CHERI-MIPS, complete enough to boot operating systems [6].

What is generated here is telling: simulators, test generators, and theorem prover input. The specification is not a convenience for producing boilerplate; it is the definition of the artefact, and the implementations are the things checked against it.

4.2 Model-driven engineering

Model-driven engineering, and its OMG-standardised form MDA, is the tradition with the largest research literature and the worst reputation. The ambitious version — platform-independent models transformed automatically into production systems — did not arrive. The recorded reasons are consistent across studies: round-trip synchronisation between model and code proved painful, platform-specific detail was too numerous and too mobile to abstract away, and large permissive metamodels were interpreted differently by different tools.

The empirical picture is more interesting than the reputation. Whittle, Hutchinson and Rouncefield surveyed 450 practitioners and interviewed 22 more, and reported that MDE

“may be more widespread than commonly believed, but developers rarely use it to generate whole systems; rather, they apply it to develop key parts of a system often using domain-specific modeling languages developed specifically for the purpose.” [7]

Two findings follow from this that bear on the rest of the paper. The first is that the modal successful application is small, local and bespoke — not a platform, but one subsystem with a DSL written for it. The second, from the same research group’s industrial case studies, is that success and failure are governed more by organisational and social factors than technical ones [8]. Tool maturity was widely reported as unsatisfactory, and productivity results were mixed in both directions.

4.3 Interface definition languages

The most widely deployed form of the mechanism is the one nobody calls a paradigm. Protocol Buffers, gRPC, OpenAPI, Smithy and WIT all take a schema as authoritative and generate clients, servers, serialisers and documentation from it, usually for several languages at once. The motivation is interoperability rather than proof: two parties who cannot read each other’s source can agree on a schema.

This tradition is invisible as a paradigm precisely because it succeeded. It is not adopted as a methodology; it arrives as a property of a framework, and the generated directory is treated as build output rather than as a model.

4.4 In-project code generation

Between the schema and the model sits the case that is least written about: a project defines its own small declarative language for one internal concern and generates several artefacts from it at build time. LLVM’s TableGen is the mature example. Instruction definitions are written declaratively in `.td` files, and backends extract from them to emit C++, documentation and command-line option tables — explicitly, in the project’s own description, so that “we can produce many things based on that single source file” [9].

The stated purpose is the removal of redundancy across near-identical declarations. There is no proof obligation and no interoperability requirement. The specification exists because writing the same fact in six places is a maintenance hazard, and that is a sufficient reason on its own. This is the tradition the case study in Section 9 belongs to, and the one Whittle’s finding predicts is both common and under-reported.

4.5 Agent-oriented specification-driven development

The 2025 wave described in Section 3.2 is the newest tradition and the only one in which the generator is non-deterministic. This is a substantive difference rather than an implementation detail. In the other four, regenerating from an unchanged specification produces byte-identical output, which is what makes the generated tree checkable in continuous integration. A generator that may produce different code from the same specification cannot offer that, and the consequences for review and for regression testing are still being worked out.

5. How common is it?

Prevalence depends entirely on which tradition is counted, and the honest composite answer is that the mechanism is near-universal while the methodology is rare.

Interface definition is the most measurable. Postman’s 2025 State of the API report puts 83.2 per cent of respondents at some level of API-first adoption, with 25 per cent describing themselves as fully API-first — up from 74 per cent and 66 per cent reporting “API first” in the two preceding years [10]. These are self-reported figures from a vendor survey of a self-selected population, and “API-first” is a broad label; they are useful as a direction of travel rather than as a measurement. Even discounted heavily, they describe a practice that is normal rather than exotic.

Model-driven engineering is the tradition where the reputation and the evidence diverge most sharply. The survey position — more widespread than believed, but applied to parts rather than wholes, frequently through purpose-built DSLs [7] — means published adoption figures systematically undercount it, because the practitioners are not using a tool with a name that a survey would ask about. A team that writes a YAML file and a generator has done MDE, and will not say so.

In-project generation is, for the same reason, effectively uncouned. It has no vendor, no conference track and no survey instrument. Its prevalence can only be inferred from artefacts — the `zz_generated` prefixes, the `.td` files, the “do not edit” banners — which are ubiquitous in infrastructure code and invisible in adoption statistics.

Agent-oriented SDD is too young for prevalence data of any quality. The investment is real and the tooling is being used, but at the time of writing the only disinterested assessment available is an “Assess” rating with usability caveats.

6. Which languages?

The choice of implementation language correlates with the tradition, not with the paradigm — and, more usefully, with one property of the ecosystem.

Tradition	Specification written in	Target languages
Formal refinement	Isabelle/HOL, Coq, Sail, ASL	C, Haskell, OCaml
Model-driven engineering	UML, EMF/Ecore, bespoke DSLs	Java (Eclipse tooling); C from Simulink in embedded
Interface definition	Protobuf, OpenAPI, Smithy, WIT	Deliberately many: Go, Java, C++, Python, TypeScript, Rust
In-project generation	Project-specific DSLs	C++ (TableGen), Go (Kubernetes), Rust
Agent-oriented SDD	Structured natural language	Agnostic in principle; TypeScript and Python in practice

The pattern worth extracting is that the determining factor is whether a language’s ecosystem has a first-class, conventional place for build-time generated source. Go has it culturally, through `go generate` and a strong convention of committing generated files with a marker comment. Rust has it structurally, through `build.rs` and module inclusion. C++ acquired it project by project, TableGen being the best-known instance. Where that convention is absent, generated code tends to be either checked in and then hand-edited — which destroys the property the generation existed to provide — or regenerated inconsistently across developer machines.

No language in the survey is intrinsically better suited than another. What differs is how much ceremony the ecosystem imposes on the generated tree, and therefore how likely a maintainer is to keep the discipline when it becomes inconvenient.

7. A cross-cutting axis: the direction of derivation

“Single source of truth” is stated as though the source were always a human-written description from which machine artefacts flow. In practice the derivation runs in both directions, and which direction a project chooses changes what can go wrong.

Kubernetes is the instructive counter-example. Its API types are Go structs annotated with markers such as `+k8s:deepcopy-gen`, and the generators produce deep-copy implementations, typed clients and — notably — the OpenAPI schema itself [11]. The schema is an output. The authoritative statement of the API is the annotated Go source, which is also the implementation.

This is *code-first*, and it inverts the trade. A code-first tree cannot suffer the transcription failure described in Section 8, because there is no transcription step: the implementation is the specification. What it gives up is the ability to state the rule in a vocabulary independent of the language it is implemented in, and with it the ability to generate an implementation for a second language, or to review the rule without reading Go.

Schema-first — `xtgeoip`, `protobuf`, `OpenAPI-as-contract` — buys that independence and pays for it with a transcription boundary. Neither direction is correct in general. The relevant observation is that the phrase “single source of truth” describes both, and conceals the difference between them.

8. The shared failure mode

8.1 What derivation guarantees

The guarantee a derived tree provides is precise and worth stating exactly, because it is narrower than the slogan. If every artefact is generated from one description by a deterministic generator, then no two artefacts can disagree with each other. Documentation cannot drift from behaviour; the test matrix cannot assert something the parser does not do; the error message in the manual cannot be one the binary no longer emits. Drift is not merely discouraged, it is inexpressible.

This is a real and substantial property. Nearly all of the maintenance burden that motivates the mechanism — in TableGen, in protobuf, in a project's own build-time DSL — is drift, and drift is genuinely eliminated. Nothing in what follows should be read as diminishing that.

8.2 What it does not guarantee

The guarantee is *conditional* on the specification, and says nothing about it. Every artefact in the tree agrees with the root because it was computed from the root. If the root is wrong, the tree is uniformly, consistently, provably wrong — and the property that makes the architecture valuable is exactly the property that removes the evidence.

Consider what normally reveals a defect of this kind. In a hand-maintained codebase the manual page and the parser are written by different people at different times in different vocabularies. When they disagree, someone notices. That disagreement is noise — it is the drift the architecture exists to abolish — but it is also *signal*, and abolishing the noise abolishes the signal with it. Worse, the defect is not merely undetected but amplified: after regeneration, six artefacts assert the wrong rule where previously one did, and each of them will be cited in good faith as confirmation of the others.

The boundary this exposes is the authoring step. Derivation guarantees fidelity *downstream* of the specification and guarantees nothing about fidelity *into* it. The root of the tree is hand-written, and it is the one artefact in the system that nothing checks.

8.3 The N-version precedent

This structure is not new, though it does not appear to have been named for derived artefact trees. It is the mirror image of a result established forty years ago in fault-tolerant computing.

N-version programming proposed that several teams independently implement the same specification, with a voter selecting among their outputs at run time. The reliability argument depended entirely on the versions failing independently. Eckhardt and Lee showed theoretically that such independence cannot be assumed as a matter of principle [12], and Knight and Leveson tested it: 27 versions, written independently at two universities, subjected to one million inputs. The individual versions were highly reliable, but the number of inputs on which more than one version failed was substantially greater than independence would predict [13]. Independence of failure could not, in general, be assumed.

A significant part of the reason is that the versions shared one specification. Where a specification is silent, ambiguous, or wrong, every implementation derived from it inherits the same defect, and no amount of implementation diversity recovers from a common-mode fault in the thing they all read.

A specification-driven tree reaches the same place from the opposite direction. N-version programming assumed an independence it did not have; a derived tree discards independence deliberately, and receives consistency in exchange. That is a defensible and usually a good trade — consistency is what most projects actually need, and machine-checked reliability is not on offer at this price anyway. But the trade should be made with the consequence in view: the tree inherits the known common-mode failure without the mitigation, and it has one specification rather than N, which is the maximally correlated case.

8.4 Not a validation failure

It is tempting to file this under Boehm’s distinction between verification (“are we building the product right?”) and validation (“are we building the right product?”) [14], and to conclude that generation buys the former and never claimed the latter. That framing is close but not exact, and the imprecision matters.

In the case examined in Section 9, nobody misunderstood the requirement. The correct rule was written down, correctly, in two separate documents. It was lost in transcription *into* the specification file. That is not a failure to determine what the product should do; it is a fidelity failure at a translation boundary, of exactly the kind that a compiler exists to prevent and that no compiler was watching here.

The sharper statement is therefore not “generation does verification, not validation,” which invites the reply that validation was never its job. It is that a derived tree **verifies every edge except the first one**, and the first edge — human intent into machine-readable specification — is where a transcription error is both most likely and least visible.

9. Case study: a four-month agreement on a wrong rule

The following is drawn from the authors’ own project, a Rust utility that builds binary geolocation databases. It is a small instance of tradition 4.4: one declarative file, `cli.yaml`, defines the command-line semantics, and a generator emits the guard table, the error constants, the manual page, two documentation formats and the integration test corpus from it.

9.1 The defect

The tool accepts modifier flags whose meaning depends on context. One of them, `-p` (“prune”), removes old archived files, and which family of files it removes depends on what the invocation produced. In the `run` context the program performs a remote fetch — producing a new compressed source archive — and, if `-b` is given, also produces a new binary backup. Both land in the same directory. The combination `run -b -p` therefore offers `-p` two candidate targets and no way to choose between them.

The manual page said so. It had said so since it was written:

“Using `-p` in a context where both fetch and backup are active (e.g. `run -b -p`) is an error: the prune target is ambiguous. To prune both, run the program twice.”

The specification disagreed. It declared the same invocation valid, with a determinate four-step plan, and the guard intended to reject the ambiguity required three flags where two were sufficient — it demanded a `-c` (“clean”) flag that has nothing whatever to do with choosing a prune target. The binary accepted `run -b -p` and silently pruned only one of the two families.

9.2 Why nothing caught it

The project is not short of checks. It has an exhaustive snapshot test pinning the parser’s behaviour over all 136 flag combinations; a generation-time cross-check proving the documented examples agree with the guard table; a contradiction test suite; and 193 unit tests. Every one of them passed throughout.

They passed because they agreed. The specification said the combination was valid; the guard table generated from it accepted the combination; the example matrix generated from it documented the combination as valid; the manual’s example section, generated from it, listed the combination among the working invocations; the test corpus, generated from it, asserted success. The snapshot test recorded what the parser did — which was what the specification said — and locked it in as the expected value. A snapshot pins what a function *is*, not what it *ought to be*, and so a wrong premise is not caught by one; it is ratified by one and thereafter defended by it.

Six artefacts asserted the wrong rule. Not one of them constituted independent evidence for any other, because all six were computed from the same file.

9.3 What did catch it

Two artefacts dissented, and both were hand-written prose that no test read. The first was the manual page template quoted above. The second, discovered during the investigation, was an enumeration of command-line behaviour pre-dating the specification file entirely, which states the rule in a different notation: `when: has: [b, p]` — precisely the two-flag predicate that was eventually shipped as the fix.

This second document settles the character of the error. The intent was recorded correctly, in a rule-shaped form, before the specification was authored. The specification did not embody a different design decision; it embodied a mis-transcription of an existing one, and the third flag entered because the guard was patterned after a neighbouring guard where all targets happen to be flag-driven. In `run`, one target is implicit — a remote fetch produces a new archive whether or not any flag requested it — and the pattern silently did not transfer.

9.4 What the architecture did well

The corrective was correspondingly cheap, and this deserves equal emphasis. The behavioural change was one line: the guard's required flag set. Regeneration propagated it to six artefacts with no hand-editing. The blast radius was predictable before the edit and confirmed exactly after it — two rows of the 136-combination snapshot moved and nothing else did. Guard precedence required no reasoning, because the snapshot proved it rather than the author asserting it.

The generation step also refused an incoherent intermediate state. The specification carries a proof obligation that each error case be demonstrated by exactly one example, since that example is the sole declared link between an error identifier and its message text. Tightening the guard caused two examples to claim the same error, and code generation aborted rather than emit the ambiguity — in a change whose subject was ambiguity. The forced resolution was the correct one.

The manual cost of the change fell entirely on what was *not* yet derived: three hand-maintained corpus-size assertions in the integration test binary. That is the accurate price of a semantic change under this architecture — proportional to the un-derived remainder, not to the size of the change.

10. Practical consequences

Four recommendations follow, stated in decreasing order of confidence.

- **Keep an independent restatement of the rules, and do not treat it as duplication.** A hand-written document that states the same rule in a different vocabulary is the only oracle a derived tree can have for its own root. The instinct of a single-source-of-truth architecture is to delete such documents as redundant, or to generate them and thereby fold them into the tree. Both moves remove the last independent witness. In the case study, the two artefacts that were right were precisely the two that nothing generated and nothing checked.
- **Check the root against the restatement, not just the tree against the root.** Consistency checks naturally grow downward, because the derived artefacts are machine-readable and easy to compare. The check that was missing in the case study was upward: nothing compared the manual's prose claims about invalid combinations against the guard table that implements them. This is harder than the checks that existed — prose is not a structured list — which is why it had not been written, and why the defect survived a deliberate audit of the manual page conducted two days earlier.
- **Treat agreement among generated artefacts as one observation, not many.** A review that finds the schema, the documentation and the tests all in accord has confirmed one fact: that the generator is deterministic. It has not corroborated the rule. This is worth stating explicitly in review guidance, because the psychological pull of six agreeing documents is considerable and entirely unearned.
- **Prefer specifications that are hard to mis-transcribe.** The defect in the case study was a predicate with one term too many, patterned after a neighbouring rule whose shape did not transfer. Specification formats that force the author to state *why* a rule holds — or that derive the rule from the structure it constrains, rather than restating it — reduce the surface on which this error is possible. This is the least evidenced of the four recom-

and is offered as a direction rather than a technique.

11. Gaps in the literature

Three gaps became apparent while assembling this survey.

The common case is the least studied. Formal refinement and model-driven engineering have substantial literatures. Interface definition has extensive practitioner documentation. The in-project build-time DSL — one project, one small language, several generated artefacts, no vendor and no methodology — has almost none, despite the survey evidence that this is where most successful model-driven practice actually resides [7]. The absence is self-reinforcing: because the practice has no name, its practitioners do not identify as a community, and nothing is written for them.

The failure mode has a theory in the wrong field. Common-mode failure arising from a shared specification is well understood in fault-tolerant computing [12][13], and appears not to have been transferred to derived artefact trees, where the correlation is total rather than partial and therefore stronger. The recent literature on code and tests generated together by a language model has independently rediscovered the same principle — that an oracle derived from the generator is not an oracle — which suggests the general form is worth stating once, in terms that cover all three cases.

Direction of derivation is under-theorised. Schema-first and code-first architectures are routinely compared on ergonomics and tooling, and rarely on what each makes impossible. A code-first tree cannot suffer a transcription failure into the specification, because there is no such step; it pays for that with the loss of language-independent expression of the rule. This is a genuine trade with consequences for review, portability and failure modes, and it is largely absent from the discussion that the phrase “single source of truth” is used to close.

12. Conclusion

Specification-driven architecture is a real and widely practised mechanism with five separate lineages, no unified literature, and a name that currently denotes the youngest and least proven of the five. Its central guarantee — that derived artefacts cannot disagree — is genuine and valuable, and is narrower than it is usually taken to be. It holds downstream of the specification and nowhere else.

The specification itself is written by a person, checked by nothing, and believed by everything. Where that authoring step goes wrong, the architecture does not merely fail to help; it removes the disagreement that would have revealed the error and multiplies the artefacts that assert it. The mitigation is not more derivation. It is deliberately retained redundancy: a hand-written statement of the same rules in a different vocabulary, and a check that compares the two. That is an unfashionable conclusion for a single-source-of-truth architecture to reach, and it is what the evidence supports.

References

- [1] Ostroff, J. S., Makalsky, D., and Paige, R. F. “Agile Specification-Driven Development.” *Extreme Programming and Agile Processes in Software Engineering (XP 2004)*, LNCS, Springer-Verlag, June 2004.
- [2] Thoughtworks. “Spec-driven development.” *Technology Radar*, Vol. 34, November 2025. Assess ring.
- [3] Klein, G., et al. “seL4: Formal Verification of an Operating-System Kernel.” *Communications of the ACM*. See also Cock, D., Klein, G., and Sewell, T., “Refinement in the Formal Verification of the seL4 Micro-kernel,” and the seL4/l4v specification and proof repository.
- [4] Reid, A. “ARM Releases Machine Readable Architecture Specification.” ARMv8.2-A specification in ASL, executable and validated against ARM’s own architecture conformance tests.
- [5] RISC-V. *Sail RISC-V model*, riscv/sail-riscv. The RISC-V foundation’s standardised executable formal specification of the ISA semantics.

- [6] Armstrong, A., et al. “ISA Semantics for ARMv8-A, RISC-V, and CHERI-MIPS.” *POPL 2019*.
- [7] Whittle, J., Hutchinson, J., and Rouncefield, M. “The State of Practice in Model-Driven Engineering.” *IEEE Software*, 31(3):79–85, 2014. Survey of 450 practitioners and 22 in-depth interviews.
- [8] Hutchinson, J., Whittle, J., and Rouncefield, M. “Model-driven engineering practices in industry: Social, organizational and managerial factors that lead to success or failure.” *Science of Computer Programming*. See also *Model-driven engineering practices in industry*, ICSE 2011.
- [9] The LLVM Project. *TableGen Overview* and *TableGen Programmer’s Reference*; “Tools for Learning LLVM TableGen,” LLVM Project Blog, December 2023.
- [10] Postman. *2025 State of the API Report*. Self-reported figures from a self-selected respondent population.
- [11] The Kubernetes Project. *kubernetes/code-generator*; Red Hat, “Kubernetes Deep Dive: Code Generation for CustomResources.”
- [12] Eckhardt, D. E., and Lee, L. D. “A theoretical basis for the analysis of multiversion software subject to coincident errors.” *IEEE Transactions on Software Engineering*, SE-11(12):1511–1517, 1985.
- [13] Knight, J. C., and Leveson, N. G. “An experimental evaluation of the assumption of independence in multiversion programming.” *IEEE Transactions on Software Engineering*, 1986. 27 independently written versions, one million test cases.
- [14] Boehm, B. W. “Verifying and Validating Software Requirements and Design Specifications.” *IEEE Software*, 1(1):75–88, 1984.

Sources were consulted in September 2026. Where a secondary summary and a primary document disagreed, the primary document was followed; see Section 2.